T.C.
İSTANBUL KÜLTÜR UNIVERSITY
FACULTY OF ENGINEERING



GRADUATING PROJECT EVALUATION APPLICATION

SOFTWARE DEVELOPMENT WITH STANDARDS

Sprint 1 – WP-04

Database Design

2100003970 Bora Kaan Haydar

November 2026

1 – Introduction

The aim of this task is to design the relational database of the capstone evaluation application. The design defines the tables, columns and integrity constraints that store users, projects, jury assignments, evaluation criteria, evaluation scores and final results. It follows the Information Viewpoint of IEEE 1016 and targets the Integrity sub-characteristic of ISO/IEC 25010. PostgreSQL was selected as the database management system.

2 – Entity Relationship Diagram

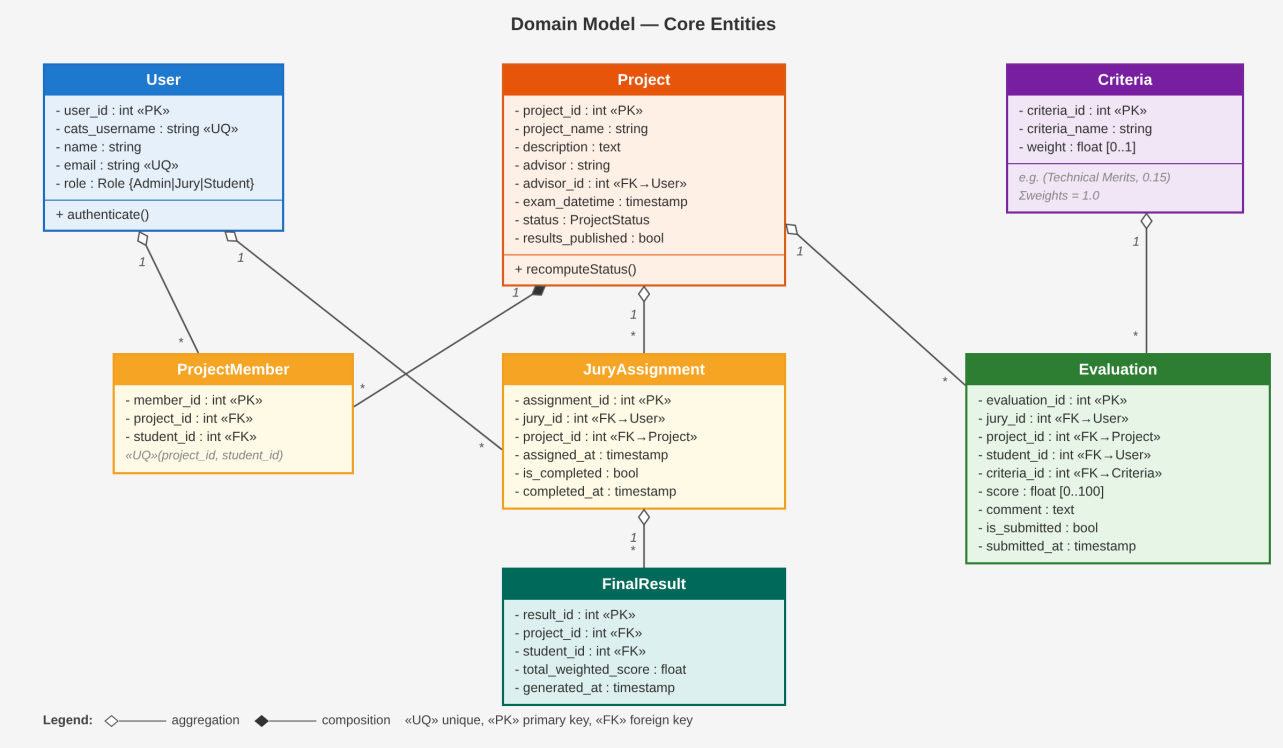


Figure 1 Domain model of the core entities

Figure 1 shows the seven core entities and their relationships. A project has one or more student members and one or more assigned jury members. Each evaluation links a jury member, a project, a student and a criterion, and final results are stored per student and project.

3 – Tables

Table	Purpose
`users`	User identity, role (Admin / Jury / Student) and CATS username
`projects`	Project information, advisor, exam date and status
`project_members`	Student–project relationship (a project may have several students)
`jury_assignments`	Assignment of jury members to projects
`criteria`	Evaluation criteria and their weights
`evaluations`	Scores per jury × project × student × criterion
`final_results`	Weighted final score calculated for each student

4 – Columns and Constraints

Table	Columns and constraints
`users`	`user_id` SERIAL PK · `cats_username` VARCHAR UNIQUE · `name` VARCHAR · `email` VARCHAR UNIQUE · `role` VARCHAR CHECK (Admin, Jury, Student)
`projects`	`project_id` SERIAL PK · `project_name` VARCHAR · `description` TEXT · `advisor_id` INT FK → users · `exam_datetime` TIMESTAMP · `status` VARCHAR CHECK (Pending, In Progress, Completed, Finalized)
`project_members`	`member_id` SERIAL PK · `project_id` INT FK → projects · `student_id` INT FK → users · UNIQUE (project_id, student_id)
`jury_assignments`	`assignment_id` SERIAL PK · `jury_id` INT FK → users · `project_id` INT FK → projects · `assigned_at` TIMESTAMP · `is_completed` BOOLEAN · UNIQUE (jury_id, project_id)
`criteria`	`criteria_id` SERIAL PK · `criteria_name` VARCHAR · `weight` NUMERIC CHECK (0 ≤ weight ≤ 1)
`evaluations`	`evaluation_id` SERIAL PK · `jury_id` INT FK → users · `project_id` INT FK → projects · `student_id` INT FK → users · `criteria_id` INT FK → criteria · `score` INT CHECK (0 ≤ score ≤ 100) · `comment` TEXT · UNIQUE (jury_id, project_id, student_id, criteria_id)
`final_results`	`result_id` SERIAL PK · `project_id` INT FK → projects · `student_id` INT FK → users · `total_weighted_score` NUMERIC · `generated_at` TIMESTAMP · UNIQUE (project_id, student_id)

5 – Integrity Rules

- **Per-student scoring:** The UNIQUE (jury_id, project_id, student_id, criteria_id) constraint ensures that each jury member scores every student of a team separately for each criterion, and that the same evaluation cannot be stored twice.
- **Score range:** Scores must be between 0 and 100. This rule is checked both in the application layer and by a CHECK constraint in the database.
- **Weights:** Each weight must be between 0 and 1, and the sum of all weights must equal 1.0.
- **Deletion behaviour:** When a project is deleted, its memberships, assignments, evaluations and results are removed with it (ON DELETE CASCADE).

6 – Evaluation Criteria

#	Criterion	Weight
1	Technical Merits	0.15
2	Project Design and Implementation	0.40
3	Report	0.15
4	Presentation	0.10
5	Demo	0.10
6	Papers	0.10
	Total	**1.00**

Criteria and weights are not hard-coded; they are stored in the `criteria` table, so the rubric can be changed without modifying the source code.